Creating a Prototype for a Data Anonymization Service

MASTER THESIS

Tobias Kaiser

Submitted on 30 July 2026

Friedrich-Alexander-Universität Erlangen-Nürnberg
Faculty of Engineering, Department Computer Science
Professorship for Open Source Software

Supervisor:
Thomas Wolter
Prof. Dr. Dirk Riehle, M.B.A.

Friedrich-Alexander-Universität
Faculty of Engineering

Declaration of Originality

I confirm that the submitted thesis is original work and was written by me without further assistance. Appropriate credit has been given where reference has been made to the work of others. The thesis was not examined before, nor has it been published. The submitted electronic version of the thesis matches the printed version.

Erlangen, 30 July 2026

License

Erlangen, 30 July 2026

Abstract

This thesis addresses the critical conflict between data-driven software engineering analytics and individual privacy rights under the General Data Protection Regulation (GDPR). The MECOIS project transforms vast development metadata into actionable productivity insights, yet the granular nature of this information poses significant privacy risks. To resolve this, this work designs and implements a prototype data anonymization service as a decoupled, containerized system within the MECOIS architecture. The service utilizes a proxy-based approach for identity management, pseudonymizing direct identifiers into deterministic Universally Unique Identifiers (UUIDs) and achieving k-anonymity by grouping contributors into organizational equivalence classes. Evaluation of the prototype indicates that it successfully maintains data integrity for high-dimensional metadata while effectively mitigating the risk of ”singling out” individuals. Ultimately, this study provides a scalable and legally grounded foundation for software analytics that respects developer privacy.

Contents

List of Figures

List of Tables

Acronyms

A4NT Adversarial Author Attribute Anonymity Neural Translation

API Application Programming Interface

BDSG Bundesdatenschutzgesetz (English: Federal Data Protection Act)

DPO Data Protection Officer

DSGVO Datenschutz-Grundverordnung

FAU Friedrich-Alexander University of Erlangen–Nuremberg

FIR functional integration requirements

FLR functional legal requirements

FSR functional system requirements

GAN Generative Adversarial Network

GDPR General Data Protection Regulation

HTTP Hypertext Transfer Protocol

JSON JavaScript Object Notation

LLM Large Language Model

NFIR nonfunctional integration requirements

NFLR nonfunctional legal requirements

NFSR nonfunctional system requirements

NLP Natural Language Processing

REST Representational State Transfer

URL Uniform Resource Locator

UUID Universally Unique Identifier

1 Introduction

Deriving actionable insights from vast datasets is an innovation driver across different industries, from healthcare analytics to financial modeling. Yet often times, these innovations stand directly at odds with data privacy, which leads to conflicting ambitions. On one hand, organizations are legally and ethically mandated to protect individual privacy under frameworks such as the General Data Protection Regulation (GDPR) and emerging global standards. On the other hand, the continuous processing and exploitation of collected data can be a powerful tool for driving financial returns, improving operational efficiency, and securing a competitive market advantage.

In the specialized field of improving the productivity of software engineers, this conflict is centered on the metadata generated by development teams through their daily interactions with tools like Git and Jira. Effectively harnessing this information to drive productivity while complying with privacy laws requires innovative frameworks that can navigate the complexities of high-dimensional data.

The MECOIS project, a research initiative from the Friedrich-Alexander University of Erlangen–Nuremberg (FAU), addresses this "Big Data" challenge within the specific domain of software engineering by transforming this metadata generated during development into usable metrics. Software development teams leave a continuous trail of activity-based artifacts in their used tools, creating a high-dimensional data set that is traditionally difficult to interpret due to overlapping metrics, complexity, and incompatible value ranges. MECOIS solves these issues by applying statistical methods to group correlating metrics into key dimensions. This data-driven approach attempts to provide an objective Productivity Index that allows organizations to compare performance across teams and projects, identifying early-stage bottlenecks like miscommunication or resource gaps while uncovering the best practices of high-performing teams. (Riehle, 2025)

To address the tension between data utilization and privacy, this thesis proposes a prototype for a dedicated anonymization service designed to process the metadata collected within the MECOIS framework. The granular nature of this data, including timestamps, commit details, and developer identities, presents significant privacy risks. The proposed service aims to anonymize this data in strict accordance with German law, ensuring that the trail of artifacts used to calculate the MECOIS Productivity Index is stripped of personally identifiable information. Integrating this anonymization layer enriches MECOIS's mission of empowering software developers through data-driven analysis with a privacy-first approach that meets modern legal and ethical standards.

2 Key Terms and Definitions

2.1 German Privacy Laws

In Germany data privacy is regulated in two laws: The Datenschutz-Grundverordnung (DSGVO) and the Bundesdatenschutzgesetz (English: Federal Data Protection Act) (BDSG).

2.1.1 General Data Protection Regulation

The DSGVO (often also abbreviated to DS-GVO) is the German name for the GDPR, which is a European Union regulation designed to ensure the protection of personal data and the free flow of data in Europe. It has been mandatory in all EU member states since May 25, 2018, and applies equally to private companies, public authorities, associations, and individuals. The GDPR strengthens the rights of EU citizens, as they now enjoy the same legal certainty in all member states and can decide how their own data is processed. For companies, it simplifies cross-border activities through uniform rules of conduct and the so-called "one-stop-shop" procedure, but also provides for substantial fines in the event of violations. While it is directly applicable in all EU member states, individual countries still have their own national data protection laws to supplement it. (für den Datenschutz und die Informationsfreiheit [BfDI], 2025)

2.1.2 Federal Data Protection Act

Germany passed its own privacy laws in 1977 called the BDSG. It had to be revised upon the adoption of the GDPR. Today the BDSG and DSGVO complement each other, since the GDPR has escape clauses, where the BDSG steps in. This applies, for example, to the processing of employee data or the establishment of supervisory authorities. (BfDI, 2025) (Däubler, 2017, pp. 68–70)

2.2 Pseudonymization and Anonymization

To comply with the data privacy laws, data often has to be pseudonymized or anonymized.

2.2.1 Pseudonymization

According to Article 4 No. 5 of the GDPR, pseudonymization is defined as the processing of personal data in such a manner that it can no longer be attributed to a specific data subject without the use of additional information. This additional information, often referred to as a ”key” for re-identification, must be kept separately and be subject to rigorous technical and organizational measures to ensure that the personal data is not assigned to an identified or identifiable natural person. Crucially, pseudonymization is characterized as a processing operation rather than a static state, involving the active transformation of cleartext data into pseudonyms to mitigate risk while maintaining the utility of the data set. Because the underlying connection to a natural person is merely obscured rather than permanently severed, pseudonymized data remains classified as ”personal data” and thus stays fully within the legal scope and obligations of the GDPR. (Members of the ”Deutsche Gesellschaft für Medizinische Informatik et al. [GMDS], 2024, pp. 8, 21, 71) (Schwartmann et al., 2022, p. 14)

2.2.2 Anonymization

In contrast, anonymization is the process of transforming personal data into anonymous information so that the data subject is no longer identifiable. While not explicitly defined within the text of the GDPR itself, it is recognized as a process that results in data that does not relate to an identified or identifiable natural person. A successful anonymization is fundamentally irreversible; once the process is complete, a re-identification of the individual must be impossible for the controller or any third party. Within the academic and legal discourse, a distinction is often made between ”absolute anonymization,” where re-identification is factually impossible for everyone, and ”factual” or ”relative anonymization,” where re-identification would require a disproportionate effort in terms of time, cost, and labor. Once data is effectively anonymized, it is no longer considered personal data, and the requirements of the GDPR cease to apply. (GMDS, b., BvD, 2024, pp. 1, 9–10, 13, 21, 68, 73) (Schwartmann et al., 2022, pp. 3, 16–19, 23)

2.2.3 Difference Pseudonymization and Anonymization

The fundamental difference between the two terms lies in the reversibility of the process and the subsequent legal classification of the data. Pseudonymization is a reversible procedure where the link between the data and the individual is preserved via a separate key, meaning the data retains its character as ”personal data”. Anonymization, however, aims for an irreversible state that permanently breaks this link, thereby removing the data from the regulatory framework of the GDPR. Furthermore, pseudonymization requires the controller to implement strict silos and access controls to manage re-identification information, whereas anonymization requires the permanent deletion or sufficient generalization of all direct and indirect identifiers to ensure that no such ”key” exists to restore the original identity. (GMDS, b., BvD, 2024, pp. 8, 13, 31, 68) (Schwartmann et al., 2022, pp. 3, 19)

2.3 RESTful API

Representational State Transfer (REST) is a fundamental architectural style that governs the design of modern web services and facilitates seamless communication between software components. A RESTful Application Programming Interface (API) operates based on a set of core principles, most notably the client-server architecture, which enforces a strict separation of concerns by delegating user-interface responsibilities to the client and data management to the server. A critical requirement of this architecture is statelessness; each request from a client must be self-contained, carrying all necessary information for the server to process it without relying on stored session context. Furthermore, RESTful systems utilize a uniform interface where resources, the central entities of the API, are identified by unique Uniform Resource Locators (URLs) and interacted with through standard HTTP methods such as GET, POST, PUT, and DELETE. By adhering to these conventions, alongside principles like cacheability and layered system design, RESTful APIs provide a simple, scalable, and predictable framework for building distributed web applications. (Selvaraj, 2024, pp. 1–14)

3 Literature Review

This section reviews existing scientific publications to establish a baseline for data anonymization services within software engineering productivity metrics.

3.1 Data Protection Laws

While extensive literature exists on the GDPR and data anonymization, existing studies typically address these concepts at a high level or focus primarily on surveillance applications. Consequently, domain-specific literature directly aligned with the scope of this research remains scarce.

Däubler (2017) is considered a definitive standard work on employee data protection. The author addresses key legal and practical issues at the intersection of labor law and data protection under the EU GDPR and the supplementary BDSG. Topics covered include the permissibility of workplace monitoring measures, such as email monitoring or video surveillance, the relationship between data protection and corporate compliance, and any prohibitions on the use of evidence in labor court proceedings. Particular emphasis is placed on the actions required of employers and co-determination bodies when drafting and amending company and service agreements.

In the context of data protection and research, the operationalization of de-identification procedures under the DSGVO is addressed by two central German guidelines that provide both technical and legal frameworks. Schwartmann et al. (2022) establishes a structured procedural model that distinguishes between theoretical ”absolute anonymity” and the more practically relevant ”factual anonymization,” where re-identification is considered impossible due to the disproportionate effort required in terms of time, cost, and manpower. Complementing this, GMDS, b., BvD (2024) focuses specifically on the sensitive domain of health data under Art. 9 GDPR, highlighting the necessity of aligning national definitions with the broader European legal framework provided by Directive (EU) 2019/1024. While Schwartmann et al. (2022) categorizes technical methods—such as randomization (e.g., differential privacy) and generalization (e.g., k-anonymity)—into specific application classes like AI algorithm training or software testing, GMDS, b., BvD (2024) provides detailed risk assessments for specific attack scenarios, including ”singling out,” inference, and linkage attacks. Both sources emphasize that anonymity is not a static state but a dynamic process that must be continuously evaluated against the evolving ”state of science and technology”. Furthermore, they converge on the legal requirement that both anonymization and pseudonymization constitute ”processing” activities under the GDPR, therefore requiring a valid legal basis and remaining subject to the controller’s accountability obligations.

The monograph Krebs and Hagenweiler (2022) provides a comprehensive examination of data protection techniques tailored for modern, data-driven environments. The authors systematically bridge the gap between legal requirements under the GDPR and BDSG and the technical implementation of protective measures. The work categorizes attributes into direct, indirect, and sensitive identifiers to address risks such as singling out, linkage, and inference. Beyond traditional methods like k-anonymity and generalization, the book explores advanced privacy-preserving paradigms including differential privacy, synthetic data generation via Generative Adversarial Networks (GANs), and federated machine learning. By addressing the specific challenges of both structured and unstructured data — such as text and multimedia — the authors offer a dual perspective on the legal and technical ”building blocks” necessary for maintaining privacy in the era of high-dimensional Big Data analysis.

3.2 Identity Management

The MECOIS project uses the tool SortingHat from the tool set GrimoireLab by CHAOSS for managing identities.

Dueñas et al. (2021) present GrimoireLab, an integrated and modular open-source tool set developed to facilitate the comprehensive retrieval, storage, and visualization of data from software repositories. This framework addresses a critical gap in software analytics by providing automated support for approximately 30 different data sources, including version control systems like Git, issue trackers such as Jira and Bugzilla, and communication platforms like Slack and mailing lists. Central to its architecture is a multi-stage pipeline that employs specialized components: Perceval for uniform data retrieval via JavaScript Object Notation (JSON) documents, Graal for detailed source code analysis, and SortingHat for advanced identity management and cross-platform contributor merging. Furthermore, GrimoireLab utilizes a distinct storage model that maintains ”raw” copies of original data alongside ”enriched” indexes optimized for visualization, a strategy that significantly enhances data maintainability, traceability, and research reproducibility. By providing a mature, field-tested framework for both academic research and industrial application, the authors demonstrate that GrimoireLab reduces the manual effort required for empirical software engineering studies while enabling the longitudinal tracking of developer activity and project health.

SortingHat is the specialized identity management component within the GrimoireLab framework, specifically designed to address the challenge of contributor fragmentation across diverse software repositories. Because developers frequently utilize different usernames, aliases, and email addresses across platforms such as Git, Slack, and Jira, SortingHat provides the essential infrastructure to unify these disparate identities into unique, individual profiles. The component operates by maintaining a dedicated relational database that stores identity data, affiliation tags, and the original source of each identity to ensure research traceability. To maintain high data quality, SortingHat utilizes a conservative algorithmic approach to merging—avoiding false positives caused by shared generic accounts—which is further complemented by HatStall, a web-based interface that allows researchers to manually curate identities and manage organizational affiliations. This process of identity unification is fundamental to the broader GrimoireLab pipeline; it allows the analytics engine to enrich data points with a unique contributor Universally Unique Identifier (UUID), thereby enabling the longitudinal tracking of developer trajectories and cross-platform activity metrics that would otherwise remain siloed. (Dueñas et al., 2021)

4 Requirements

This chapter defines the requirements necessary to establish a productivity benchmarking system compliant with the GDPR and BDSG. Following standard IEEE definitions, requirements are divided into functional, defining what the product must do, and nonfunctional, specifying how the system should operate ('ISO/IEC/IEEE International Standard - Systems and software engineering– Vocabulary', 2017). Furthermore, these requirements are structured across three main categories: legality, system architecture, and integration. Specifically, the service must abide by relevant data protection laws, adhere to author-specified system design constraints, and satisfy the integration demands outlined in the MECOIS contributing guidelines (MECOIS, 2026).

4.1 Functional Requirements

The table 4.1 specifies what the anonymization service must do. The requirements are sorted by functional legal requirements (FLR), functional system requirements (FSR), and functional integration requirements (FIR).

The requirement for Data Anonymization (FLR-1) is intentionally formulated with a broad scope, as the GDPR does not prescribe a singular technical methodology but rather establishes a high-level legal standard. Because various technical approaches can achieve compliance, Chapter 6 systematically evaluates and compares different anonymization strategies suitable for high-dimensional development metadata. This legal mandate exists in fundamental tension with the system requirement to Preserve Data (FSR-1), which seeks to maintain the integrity of all source information for analytical depth. Consequently, a primary objective of this work is to navigate this conflict by identifying a functional compromise that ensures robust individual privacy while maintaining the data utility required for the derivation of the MECOIS Productivity Index.

Table 4.1: Functional requirements

ID	Requirement	Description
FLR-1	Data Anonymization	The service must follow the German privacy laws (GDPR) and BDSG).
FSR-1	Preserve Data	The service must preserve all available Data.
FSR-2	Independent Deployment	The service can be deployed and run independent from the rest of the MECOIS project.
FSR-3	Deterministic Pseudonym Generation	The service must support consistent UUID generation across pipeline runs so that cross-platform metrics can be linked over time without revealing identity.
FIR-1	Seamless Integration	The service must integrate into the project with as few changes as possible to existing features.

4.2 Nonfunctional Requirements

The table 4.2 specifies how the anonymization service must operate. They are sorted by nonfunctional legal requirements (NFLR), nonfunctional system requirements (NFSR), and nonfunctional integration requirements (NFIR).

Table 4.2: Nonfunctional requirements

ID	Requirement	Description
NFLR-1	Anonymization Methodology Adherence	The service must implement the anonymization measures decided upon in Chapter 6.
NFSR-1	Containerized Service	The service must run in a docker container, a common used framework for containerized deployment. (Jangla, 2018, pp. 9–11)
NFSR-2	RESTful API	The service must implement a RESTful API to ensure a standardized, stateless, and scalable interface for communication between the Main System and Identity Management System.
NFIR-1	Data Integrity	The service must maintain schema consistency during the serialization and deserialization of DataFrames into JSON to ensure no data loss occurs during the HTTP transmission process.
NFIR-2	Programming Language	The Backend of the MECOIS project is written in python3. Therefore, this project must also use python3. (MECOIS, 2026)
NFIR-3	License Policy	The contributing guidelines state that dependencies should be kept to a minimum and forbids packages with a copyleft or proprietary license. (MECOIS, 2026)
NFIR-4	Styleguide and Clean Code	Implementations must follow the style guidelines of the main project. (MECOIS, 2026)

5 Architecture

The MECOIS setup consists of 3 separate systems, that communicate via HTTP (see Figure 5.1). The systems are separated to protect sensitive information.

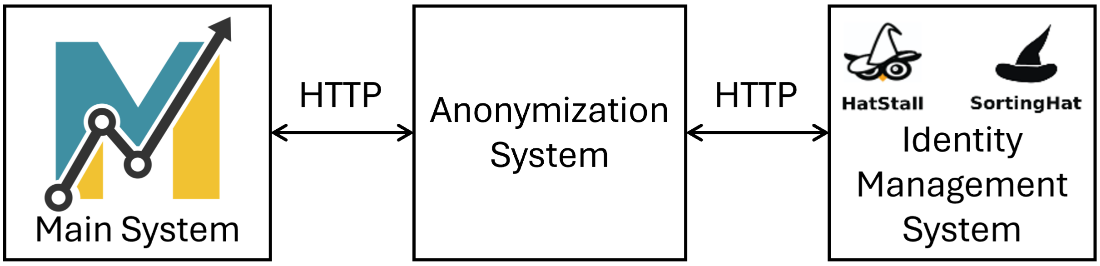

Figure 5.1: Basic System Architecture

5.1 Main System

The main system serves as the central orchestration hub of the MECOIS framework, encompassing all operational responsibilities except for identity management. It is structured as a modular environment composed of several specialized components that facilitate the end-to-end processing of software development metadata, starting from initial data ingestion to the generation of final productivity metrics.

5.1.1 Components

Figure 5.2 displays an overview of all components. (MECOIS, 2026)

Sources

The Source component serves as the primary gateway for data ingestion, defining the specific origins of the metadata processed by the MECOIS framework. Currently, the system supports a variety of platforms, including version control systems like Git (interfaced through GitHub and GitLab), documentation tools

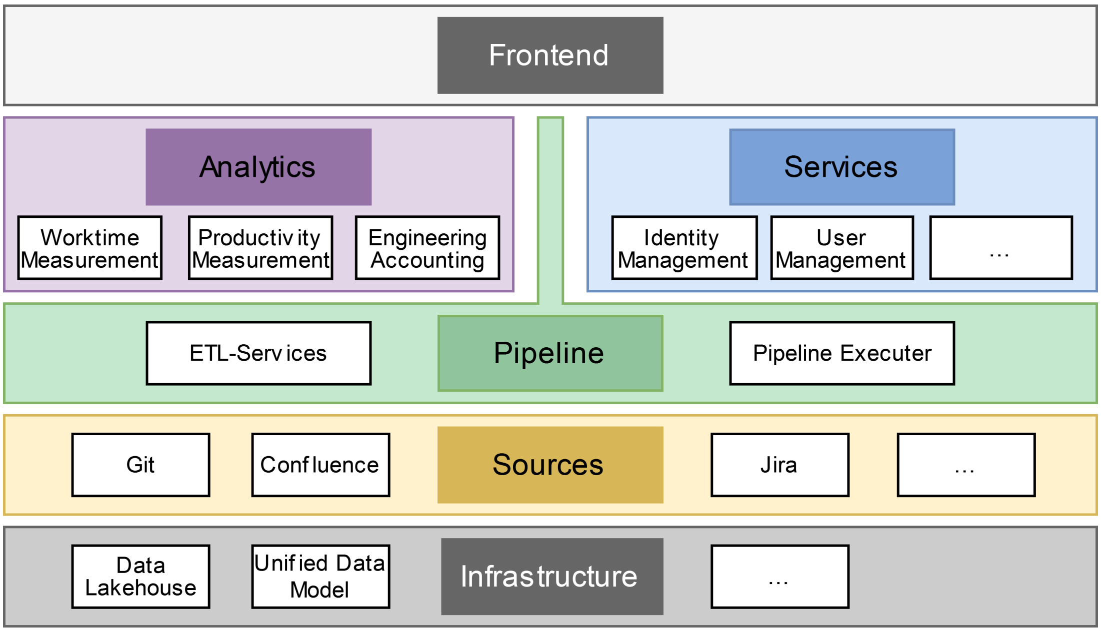

Figure 5.2: Main System Components (MECOIS, 2026)

such as Confluence, and issue trackers like Jira. This architectural layer is designed to be extensible, allowing for the integration of additional data sources in the future. For instance, when querying GitHub, the system retrieves diverse activity-based artifacts, including commits, issue logs, and pipeline execution data.

Infrastructure

The infrastructure layer provides the central data repository for the MECOIS framework, utilizing a data lakehouse architecture to manage complex metadata. A data lakehouse is a unified data architecture that combines the strengths of traditional data lakes and data warehouses, offering a high-performance and governance-friendly environment for storing raw data in diverse formats. It allows organizations to store structured, semi-structured, and unstructured data in a central location where it remains accessible for analytics, machine learning, and reporting without the need for extensive data movement. Within the MECOIS system, information progresses through four sequential stages: raw, bronze, silver, and gold. (Amazon Web Services [AWS], 2026)

Pipeline

The pipelines serve as the operational backbone of the MECOIS framework, responsible for the systematic transformation of development metadata through the various stages of the data lakehouse. Within this architecture, metadata is processed multiple times to evolve from its original, raw state into high-quality, privacy-preserving metrics. This progression is categorized into four distinct states: raw, bronze, silver, and gold. Each pipeline follows a standardized three-step execution model: first, the data is read from the data lake; second, the specific transformations are applied; and third, the results are written back to the repository.

Analytics

The final phase of the MECOIS framework involves the derivation of high-level metrics from the refined and anonymized data stored within the Silver layer. This stage, characterized as the Silver-to-Gold transformation, generates the structured analytics required for management reporting, productivity assessment, and financial documentation. By applying specialized algorithms to the processed metadata, the system transforms technical artifacts into actionable business insights that allow organizations to quantify the impact of internal collaborative efforts. Current examples of these analytical applications focus heavily on the economic and temporal quantification of developer contributions:

- **Cost Calculation in Inner Source:** As explored by Buchner and Riehle (2021), the framework can be utilized to calculate transfer prices and labor costs for Inner Source collaboration by estimating the time worked on individual commits, thereby facilitating fair cost allocation across different organizational units.
- **Contribution Time Estimation:** Building upon the previous methodology, Stenzel (2026) provides an improved approach for estimating software contribution time from Git and GitHub metadata. This work addresses real-world implementation challenges such as the disassembly of squashed commits, time zone normalization, and the elimination of statistical outliers to provide a more robust evidential basis for cost calculation.

Services

Services are helper modules for pipelines and analytics. Such helper modules are:

- AI: a framework for communicating with a Large Language Model (LLM).
- config handling: Parses configuration files for pipelines.

Front end

The front end provides a Dashboard for the user-facing components of MECOIS. Figure 5.3 shows the Data Explorer tool, where users can examine the data in the data lake. Figure 5.4 presents the Data Extractor tool, that can be used to query data from the sources and feed it to the pipelines.

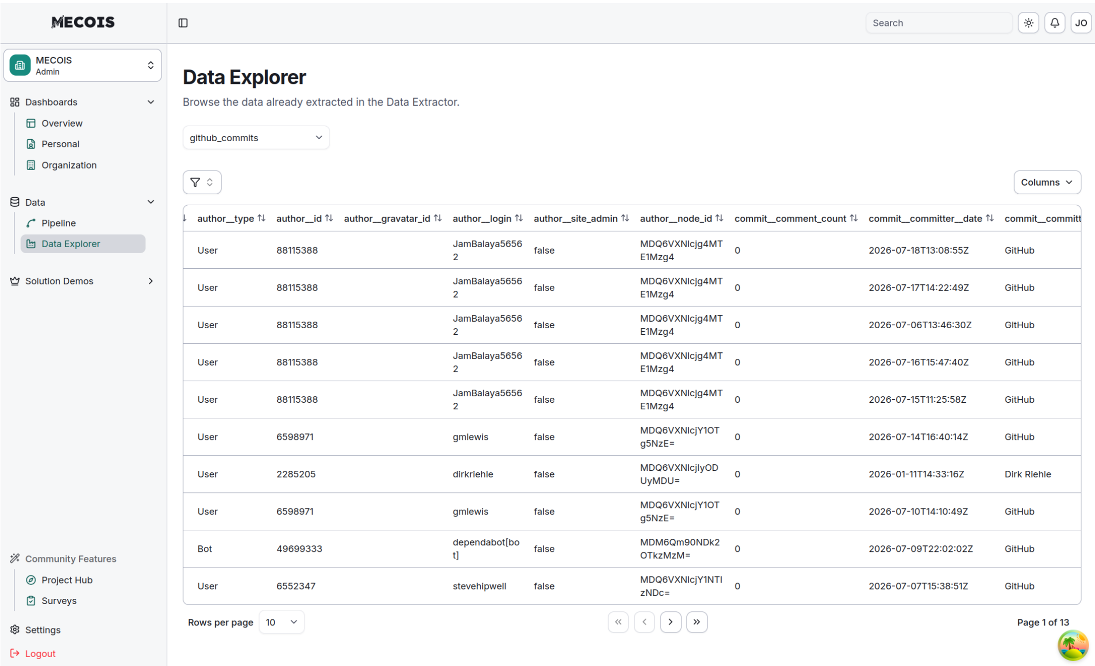

Figure 5.3: MECOIS Frontend Data Explorer

Orchestrators

The orchestrators can be used to run the pipeline. They set up the configuration and run the pipeline for the selected sources.

5.1.2 Data Flow

Upon successful extraction from the primary sources, the raw metadata is persisted within the data lakehouse in its original JSON format to ensure data provenance. This data subsequently undergoes a multi-stage refinement process, systematically evolving through a series of qualitative states defined as bronze, silver, and gold. These transitions are orchestrated by specialized pipelines, each of which adheres to a standardized three-step execution model:

1. **Data Retrieval:** The pipeline initiates by reading the required dataset from the relevant storage tier within the data lake.

2. **Transformation:** Specific analytical logic, data cleaning, or privacy-preserving transformations are applied to the records in memory.

3. **Data Persistence:** The resulting transformed and enriched records are written back to the subsequent tier of the data lake repository.

A comprehensive overview of these processing stages and the interaction between the various framework components is illustrated in Figure 5.5. (MECOIS, 2026)

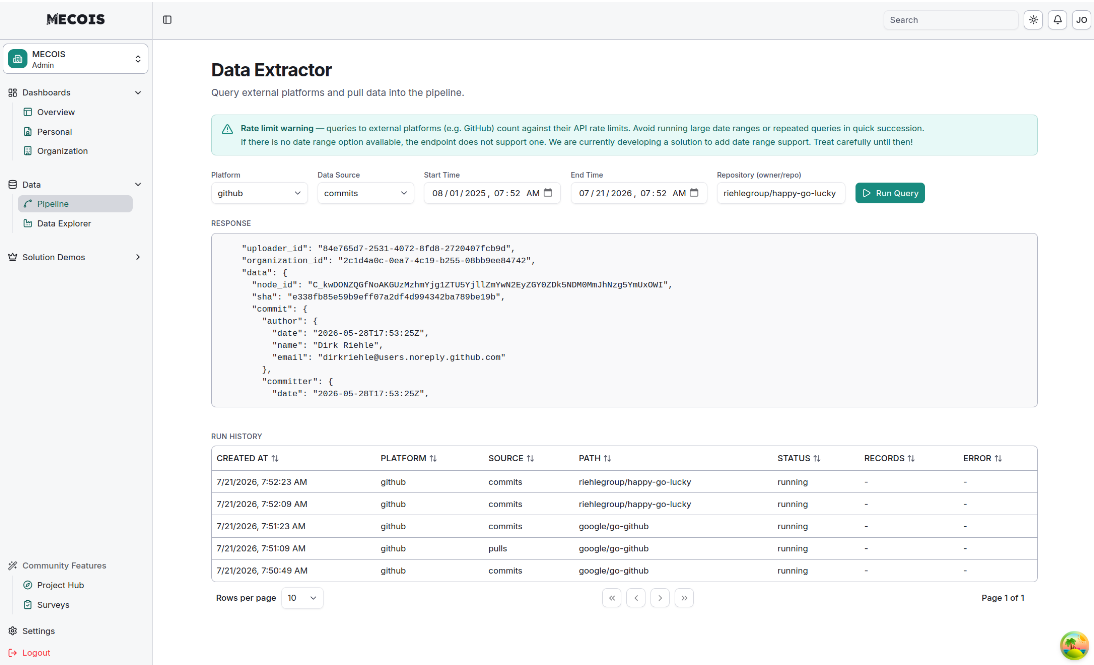

Figure 5.4: MECOIS Frontend Data Extractor

JSON to Bronze

The `JSONtoBronzePipeline` processes the raw JSON data provided by the sources and writes it unchanged to the Data lake. The data itself is not changed in any way. Each source has an own table.

Bronze to Silver

The `BronzetoSilverPipeline` serves as the primary refinement stage within the MECOIS framework, transforming raw, ingestion-tier data into a structured and privacy-compliant format suitable for high-level analytics. This pipeline executes a series of sequential transformations designed to enhance data quality, normalize schemas, and ensure adherence to legal privacy standards.

Flatten Table Because the bronze storage tier preserves the original nested structure of JSON artifacts, the data often contains hierarchical objects that are incompatible with flat relational analytics. For example, a GitHub commit object may contain nested properties for the author, which cannot be natively represented in the simplified bronze schema. To resolve this, the pipeline deconstructs these nested columns into individual, flattened columns, ensuring that each field contains only a single data point for subsequent statistical evaluation.

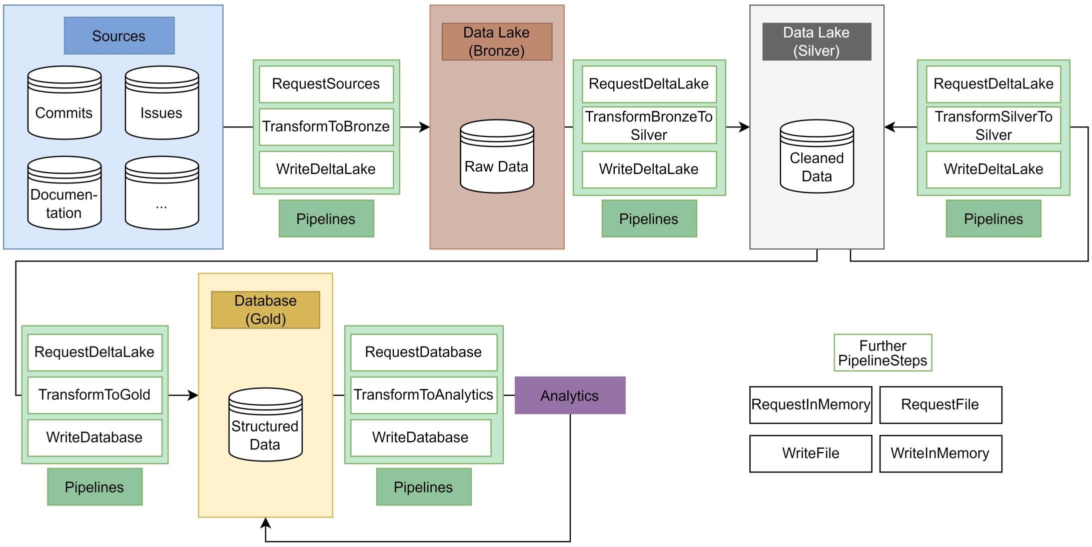

Figure 5.5: Data flow (MECOIS, 2026)

Drop Irrelevant Columns To optimize storage efficiency and focus on relevant metrics, the pipeline identifies and removes attributes that offer no analytical value to the productivity index. Metadata such as the URL for a GitHub user's avatar is discarded, which reduces the dimensionality of the dataset and ensures that only pertinent information is persisted in the silver tier.

Fix Dates Temporal metadata extracted from development sources is initially stored as unformatted strings during the bronze stage. This transformation step casts these strings into native timestamp objects, allowing the database to store them natively and facilitate efficient time-series analysis. By normalizing these attributes, the framework ensures temporal consistency and computational accuracy across all derived datasets.

Anonymize Data A central function of this pipeline is the integration of privacy-preserving measures through the decoupled anonymization service. During this phase, the data is transmitted to the service via the `change_identities` endpoint. This process ensures that the resulting silver-tier data meets the privacy requirements while remaining analytically useful.

Add UUID The pipeline generates a deterministic UUID for each record by concatenating specific columns and hashing the result.

Silver to Silver

The Silver-to-Silver processing tier is responsible for maintaining the synchronization and accuracy of the data lakehouse as contributor information evolves within the identity management system. Since the SortingHat system frequently undergoes updates—such as the merging of fragmented developer profiles or changes in organizational affiliations—the silver-tier datasets must be adjusted to reflect these new identities. To address this, the `UpdateIdentityPipeline` monitors modifications within SortingHat and triggers a re-evaluation of previously processed records in the data lake. By invoking the `update_identities` endpoint of the anonymization service, the pipeline ensures that all pseudonymized identifiers and organizational mappings remain consistent with the latest identity data, thereby preserving the reliability of the longitudinal metrics derived by the MECOIS framework.

Silver to Gold

The transition from the silver to the gold tier does not rely on a singular, monolithic pipeline. Instead, the various analytical modules themselves function as the silver-to-gold pipelines, as each specific analytic consumes the cleaned and anonymized silver data to derive specialized metrics. This stage represents the final transformation where refined metadata is converted into high-level, structured insights for the MECOIS Productivity Index.

Gold to Gold

Certain analytical applications may require a specialized mechanism to refine or update previously generated metrics. This iterative processing stage is defined as a gold-to-gold transformation, ensuring that the stored metrics can be adjusted or maintained over time without necessitating a full re-processing of the underlying silver-tier data.

5.2 Anonymization System

The anonymization system provides the anonymization service, that anonymizes the data according to the GDPR and BDSG. It provides a client library for sending data and requesting identities.

5.3 Identity Management System

The identity management system serves as the primary repository for all individual-related data within the MECOIS framework. For this purpose, the project integrates SortingHat, a specialized component of the GrimoireLab toolset developed by CHAOSS. This software is distributed under the GPL-3.0 license. To address the frequent fragmentation of identities across different platforms such as Git, Slack, and Jira, SortingHat provides a robust infrastructure for unifying disparate usernames and aliases into cohesive individual profiles. This unification process allows the system to aggregate multiple accounts belonging to a single person into a centralized entity. Furthermore, these consolidated individuals can be organized into hierarchical structures, ranging from specific teams to broader organizational units.

5.3.1 Identities

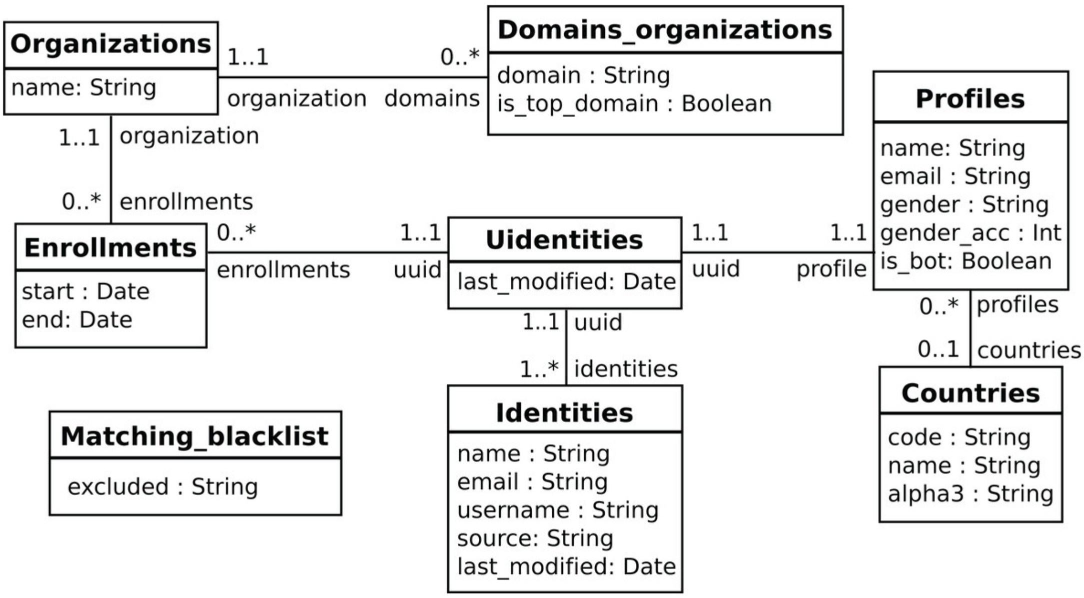

Figure 5.6: SortingHat UML diagram (Dueñas et al., 2021)

The conceptual mapping of identities within SortingHat is illustrated in the UML diagram shown in Figure 5.6. This architecture enables the system to pool diverse identities—such as various GitHub accounts or email addresses—into a single individual representation. Each unique individual is assigned a deterministic UUID, which serves as a stable identifier linked to a profile containing relevant personal information. Beyond simple identification, the system facilitates the association of these individuals with specific organizations, supporting the longitudinal tracking of developer trajectories and cross-platform activity metrics.

5.3.2 Front end Dashboard

SortingHat incorporates a dedicated web-based dashboard designed to facilitate manual identity curation and the management of contributor affiliations. This interface allows researchers and administrators to oversee and refine the unification of digital profiles, complementing the system's automated merging algorithms. (CHAOSS, 2026c) The dashboard is organized into three primary functional modules:

- **Workspace:** This area serves as a staging environment for tracking specific individual profiles that require pending administrative actions, providing a focused workflow for identity resolution. (CHAOSS, 2026c)
- **Individuals:** This module contains a centralized registry of unique profiles. Within the SortingHat framework, a profile represents a single human contributor, while identities correspond to their various accounts across disparate platforms, such as Git, GitHub, Slack, or mailing lists. This section allows for the aggregation of multiple digital aliases and email addresses into a cohesive individual representation. (CHAOSS, 2026a) (CHAOSS, 2026c)
- **Organisation:** This section manages the list of affiliations where individuals are employed or enrolled, enabling the association of contributors with their respective organizational units. (CHAOSS, 2026c)

A Screenshot of the front-end environment is provided in Figure 5.7. To comply with data privacy standards, personal identifiers within this screenshot have been redacted. An official uncensored version of the individuals board is illustrated in Figure 5.8.

5.3.3 API

The SortingHat also provides an API for automating interactions, like listing, updating and deleting individuals.

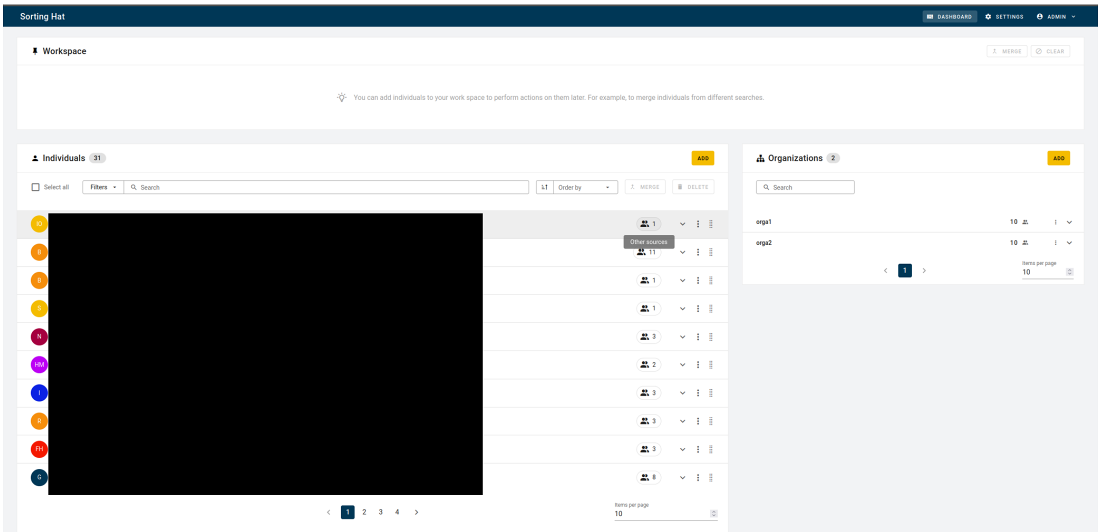

Figure 5.7: SortingHat front end (individuals are censored for privacy reasons)

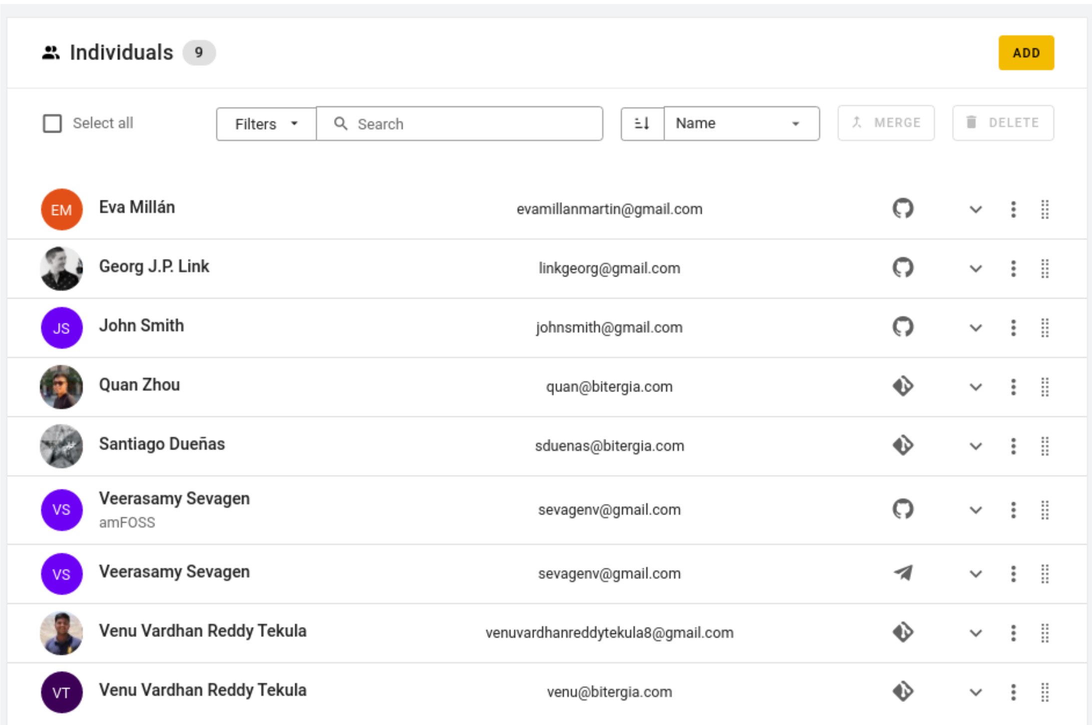

Figure 5.8: SortingHat individuals section (uncensored) (CHAOSS, 2026c)

6 Design and Implementation

Chapter 5 described the architecture of the MECOIS project and the integration of the anonymization service. This chapter will explain how the service is implemented.

6.1 Reason for an Anonymization Service

According to the BDSG, personal data may only be collected, processed, or used for a specific purpose. It is forbidden to use this date for other purposes. There are only a few specific exceptions for this rule. (Däubler, 2017, p. 72)

6.1.1 Purpose Limitation

Since software development tools like GitHub, GitLab or Jira contains personal data (e.g. git commit author name or Jira ticket reporter), its use falls under the GDPR. In most cases there is no written agreement on how the data can be used, so it can be assumed that the purpose for this data is to help developers organize their work. Generating productivity measurements for individual employees would be considered a misuse of the data. Däubler (2017, pp. 266–267) explicitly states that data that consists of operational workflows gathered by electronic systems cannot be used for measuring productivity of individual employees. (Däubler, 2017, pp. 266–267)

One exception is scientific research. If individuals are part of a scientific research project, their data can be used for this purpose. The data should get pseudonymized or anonymized as fast as possible, in order to minimize the infringement of privacy of the individual members.(Däubler, 2017, pp. 273–274)

6.1.2 Anonymized Data

The BDSG only applies to personal Data, which is defined in Section 3 (1): Personal data consists of specific information about the personal or factual circumstances of an identified or identifiable natural person (data subject). (Däubler, 2017, p. 69)

The GDPR has an similar approach. In Article 4(1) it defines personal data as any information relating to an identified or identifiable natural person (hereinafter referred to as the “data subject”). A natural person is considered identifiable if they can be identified, directly or indirectly, in particular by association with an identifier such as a name, an identification number, location data, an online identifier, or one or more factors specific to the physical, physiological, genetic, mental, economic, cultural, or social identity of that natural person. (Däubler, 2017, p. 69)

Unlike anonymized data, where re-identification is rendered impossible, pseudonymized data remains traceable to an individual. Consequently, pseudonymized data falls directly under the scope of the GDPR and BDSG. Conversely, because anonymized data no longer relates to an identified or identifiable natural person, it falls entirely outside the jurisdiction of the GDPR. (Däubler, 2017, p. 69)

6.1.3 Consequences for MECOIS

For the MECOIS project the exception for scientific research is very important regarding the development of the project. As long as MECOIS is a scientific research project, the data can be used to generate productivity measurements as long as the process serves scientific research purposes. But the data still should be pseudonymized or anonymized.

Regarding the final product, MECOIS must anonymize the Data. Otherwise it is illegal to produce productivity metrics.

6.2 Identifying Identifiers

The identification and categorization of identifiers is a fundamental prerequisite for implementing effective data protection measures, such as pseudonymization or anonymization. Attributes within a data set are typically classified into three categories: direct identifiers, indirect (or quasi-) identifiers, and sensitive attributes. (Krebs & Hagenweiler, 2022, pp. 87–88, 131–132) (Schwartmann et al., 2022, pp. 9–10) (GMDS, b., BvD, 2024, p. 37)

Direct Identifiers Direct identifiers, such as civil names, social security numbers, or email addresses, allow for the immediate identification of a natural person without further processing. (Krebs & Hagenweiler, 2022, pp. 87–88) (GMDS, b., BvD, 2024, pp. 37, 71)

Indirect Identifiers Indirect identifiers or quasi-identifiers include data points like birth dates, ZIP codes, or rare job titles that do not identify a person in isolation but can lead to identification when combined with external datasets or background knowledge — a risk often termed ”singling out”. (Krebs & Hagenweiler, 2022, pp. 87–88) (Schwartmann et al., 2022, pp. 5, 13) (GMDS, b., BvD, 2024, p. 37)

Sensitive Attributes Sensitive attributes, such as the special categories of data protected under Article 9 of the GDPR (e.g., genetic or health data), require the highest level of protection due to the potential for discrimination and significant impact on individual freedoms. (Krebs & Hagenweiler, 2022, pp. 131–133) (GMDS, b., BvD, 2024, pp. 8, 24–25)

Ultimately, the classification of any specific data point is context-dependent, as even non-identifying data may become identifying depending on the ”reasonable” means and background information available to a potential attacker. (GMDS, b., BvD, 2024, pp. 36–37, 54)

Example GitHub Commit In the context of GitHub commits, direct identifiers are: names and emails of author and committer. Since every employee has access to the GitHub repositories, indirect identifiers like the commit message (including the timestamp), ids and specific URLs can be easily resolved to the involved persons. Sensitive attributes are not present in GitHub.

6.3 Approaches for Pseudonymization and Anonymization

Techniques for pseudonymization and anonymization are categorized based on their technical approach and the type of data they address.

The boundary between pseudonymization and anonymization is fundamentally fluid because the technical methods used — such as masking, shuffling, or the variance method — are often identical for both processes. Technically, these randomization and transformation techniques serve as common building blocks; the legal and practical classification of the resulting data depends on the management of ”additional information” and the context of the data environment. For instance, a method like masking is classified as pseudonymization if the researcher retains a secure mapping table to restore the original identifiers, but it becomes anonymization if that key is irreversibly destroyed. This status remains dynamic rather than absolute, as the availability of new external datasets or advances in analytical power can enable the re-identification of data previously considered anonymous. Consequently, the transition between these two states depends heavily on the ”reasonability test”, where data is only truly anonymous if re-identification is either factually impossible or requires a disproportionate effort in terms of time, cost, and manpower. (Schwartmann et al., 2022, pp. 3–4) (GMDS, b., BvD, 2024, pp. 44–45, 55, 68, 71) (Krebs & Hagenweiler, 2022, pp. 77–80)

6.3.1 Pseudonymization Techniques

Pseudonymization involves replacing identifiers with surrogates so that the data cannot be attributed to a specific person without additional information stored separately. (Krebs & Hagenweiler, 2022, pp. 71, 77)

Masking and Replacement Direct identifiers are replaced with constant values, characters, or placeholders (e.g., replacing ”John Doe” with ”Person A” or setting the birthday to the first of January). (Krebs & Hagenweiler, 2022, pp. 77–80) (GMDS, b., BvD, 2024, pp. 40, 44)

Shuffling (Mixing) Values within a column are swapped or rearranged across different records through random distribution to remove the link between attributes and specific individuals. Direct identifiers have to be pseudonymized with another technique, to prevent users from unshuffling the data, by connecting direct identifiers. (Krebs & Hagenweiler, 2022, pp. 77–80) (GMDS, b., BvD, 2024, p. 41)

Variance Method For quantitative data, numerical values (e .g ages or dates) are altered by adding random deviations within a specified range while maintaining overall statistical distribution. (Krebs & Hagenweiler, 2022, pp. 77–80) (GMDS, b., BvD, 2024, p. 42)

Cryptographic Encryption and Hashing Cryptographic methods, specifically encryption and hashing, are essential technical measures for the pseudonymization of personal data, as they replace direct identifiers with secure surrogates. Encryption involves transforming identifiers using a secret key, making the process reversible only for authorized parties who possess that key. In contrast, hashing utilizes a one-way mathematical function to map identifiers to a fixed-length string, which is inherently designed to be non-invertible. While hashing provides a more robust barrier against re-identification than simple masking, raw hash values remain vulnerable to brute-force or rainbow table attacks if an attacker can guess the range of original inputs. To mitigate this risk, it is critical to implement a ”salt,” which is a random sequence of characters appended to the original data before it is hashed. The use of a salt significantly increases the entropy of the input, ensuring that the resulting hash is not easily predictable and preventing attackers from using precomputed tables to reverse the transformation. (Krebs & Hagenweiler, 2022, pp. 77–80) (GMDS, b., BvD, 2024, pp. 13, 42–44, 84)

Tokenization Often used in the financial sector, this replaces sensitive identifiers like card IDs with randomly generated tokens. (Krebs & Hagenweiler, 2022, pp. 77–80)

6.3.2 Anonymization Techniques for Structured Data

Anonymization aims to permanently remove the connection between data and a natural person, making re-identification practically impossible. (Krebs & Hagenweiler, 2022, pp. 73–75)

Structured Data Structured data refers to information that is organized according to a fixed schema or predefined format, ensuring that its content and meaning are clearly and uniquely determinable. It is fundamentally understood through a ”Key-Value” representation, where the ”Key” assigns specific meaning (e.g., ”First Name”) and the ”Value” contains the actual content (e.g., ”Anne”). In practice, this data is typically stored in tabular formats like spreadsheets or relational databases, where each row represents an individual data record and each column defines a specific attribute with a fixed data type, such as numeric values, categorical lists, or qualitative ratings. This rigid organization makes structured data highly searchable and suitable for systematic processing by automated analysis tools. (Krebs & Hagenweiler, 2022, pp. 45–46) (Schwartmann et al., 2022, pp. 11–12)

Randomization (Perturbation)

These methods distort attribute values to decrease the probability of identifying individuals from a dataset. (Krebs & Hagenweiler, 2022, pp. 83–86)

Stochastic Overlay Characteristics are altered with random variables so that the specific values are less accurate but general distribution is preserved. (Krebs & Hagenweiler, 2022, p. 84) (Schwartmann et al., 2022, p. 31)

Swapping A special form of stochastic overlay is swapping, where attribute values are shifted between different data records, which is particularly effective when combined with the removal of quasi-identifiers. (Krebs & Hagenweiler, 2022, pp. 84–85) (Schwartmann et al., 2022, p. 31)

Falsification (Noise Addition) Artificial statistical noise is added to some or all data points to prevent reliable estimation of an individual's original values. (Krebs & Hagenweiler, 2022, pp. 85–86) (Schwartmann et al., 2022, p. 36)

Microaggregation (Clustering) Data records are grouped by similarity, and their values are replaced by a representative average (e.g., mean or median) for that group. (Krebs & Hagenweiler, 2022, p. 86)

Generalization and Aggregation

Generalization and aggregation are the technical pillars for implementing formal anonymity models in structured data. Generalization involves coarsening attribute values, such as replacing a specific street address with a ZIP code or a precise birth date with a birth year, to reduce data granularity. Aggregation, often applied through micro-aggregation or clustering, groups individual records and replaces specific values with representative summary values like means or medians. While these techniques effectively reduce the risk of "singling out" individuals, they inherently involve a critical trade-off: increasing the level of protection leads to a corresponding loss of data utility for analytical purposes. (Krebs & Hagenweiler, 2022, pp. 86–89, 131, 138) (Schwartmann et al., 2022, pp. 33–34)

k-Anonymity The k-Anonymity model provides a standard for measuring anonymity by ensuring that every record in a dataset is indistinguishable from at least k-1 other records based on its quasi-identifiers. By grouping individuals into these "equivalence classes," the model prevents attackers from isolating a specific person within the data. While a higher value for k significantly reduces the probability of identity disclosure—calculated as 1/k—the model remains vulnerable to attacks that can derive sensitive attributes if all members of a group share the same characteristic. (Krebs & Hagenweiler, 2022, pp. 89–92, 138) (GMDS, b., BvD, 2024, pp. 44–46) (Schwartmann et al., 2022, pp. 33–34)

l-Diversity To address the limitations of k-Anonymity, the l-Diversity model requires that each k-anonymous group contains at least l "well-represented" values for sensitive attributes. This diversity ensures that an attacker cannot definitively infer an individual's sensitive information even if they know the person belongs to a specific group. Common variations include distinct, entropy, and recursive (c, l)-diversity, each offering different levels of protection against inference-based attribute disclosure. (Krebs & Hagenweiler, 2022, pp. 92–93) (GMDS, b., BvD, 2024, pp. 61–62) (Schwartmann et al., 2022, p. 34)

t-Closeness The t-Closeness criterion further refines the privacy standard by requiring that the distribution of a sensitive attribute within any group remains close to the distribution of that attribute in the overall dataset. By limiting the deviation (represented by the parameter t) between the conditional and unconditional distributions, this model prevents attackers from gaining significant knowledge about a person through group-level statistical anomalies. This is particularly useful for maintaining data utility while protecting against sophisticated inference attacks that l-Diversity might not fully mitigate. (Krebs & Hagenweiler, 2022, pp. 93–94) (GMDS, b., BvD, 2024, p. 62)

δ-Presence The δ-Presence model specifically addresses the risk of "membership disclosure," where an attacker attempts to determine whether a specific individual is included in a private dataset. This model typically operates by comparing a private dataset against a publicly available one that contains quasi-identifiers, such as a voter registry. By quantifying and limiting the probability (δ) that a person from the larger population can be linked to the specific subset, δ-Presence ensures that the mere presence of a person in a sensitive dataset remains protected. (Krebs & Hagenweiler, 2022, p. 94) (GMDS, b., BvD, 2024, pp. 62–63)

Advanced and KI-Based Methods

Differential Privacy Differential Privacy is a rigorous mathematical framework that protects individual privacy by adding controlled statistical noise to either the underlying data or query results. This perturbation ensures that the presence or absence of a single individual does not significantly alter the outcome of any statistical analysis. Unlike syntactic models, it provides a semantic guarantee of privacy that remains robust even if an attacker possesses extensive background knowledge. The level of protection is quantified by the parameter ϵ, where a smaller value indicates stronger privacy at the cost of reduced data utility. (Krebs & Hagenweiler, 2022, pp. 96–98) (GMDS, b., BvD, 2024, pp. 46–47)

Data Synthesis Data Synthesis involves building a statistical model of the original data set to generate entirely new, artificial data that reflects the properties of the original information without including real personal identifiers. Advanced techniques, such as a GAN, allow for the creation of realistic synthetic data points — such as images or medical records — that maintain the statistical distribution of the source material. This method is particularly valuable for training machine learning models or testing software when access to authentic personal data is legally restricted. While no direct link exists between the original and synthetic data, the synthesis model must be carefully steered to prevent the disclosure of unique patterns that could lead to re-identification. (Krebs & Hagenweiler, 2022, pp. 19, 99–101) (Schwartmann et al., 2022, pp. 34–35)

Semantic Anonymization Semantic Anonymization uses active ontologies and inferencing within a semantic AI-based system to transform sensitive information while preserving its analytical meaning. The process involves dividing a data set into specific "data pools" to apply separate transformation rules, a method known as semantic micro-aggregation. By focusing on the meaning and statistical relationships of data points rather than just syntactic structures, it effectively prevents re-identification through quasi-identifiers. This approach aims to maximize data utility for industrial analysis while ensuring that a person-specific trace back to the original records is impossible. (Krebs & Hagenweiler, 2022, pp. 103–107)

Federated Machine Learning Federated Machine Learning is a decentralized training approach where algorithms are trained locally on end-user devices or separate company databases rather than in a central repository. Only non-personal model parameters, such as gradients, are transmitted to a central coordinator to be aggregated into a global model. Because the raw training data never leaves its original location, this method significantly reduces the risks associated with data aggregation and unauthorized access under the GDPR. This "bringing the algorithm to the data" approach allows for the development of complex AI models across multiple partners while maintaining high data sovereignty. (Krebs & Hagenweiler, 2022, pp. 101–103) (Schwartmann et al., 2022, pp. 7, 52–53)

6.3.3 Techniques for Unstructured Data

The following techniques for unstructured data focus on removing or altering personal identifiers in formats such as text, images, and video to prevent re-identification.

Metadata Removal This process involves deleting the decoupled metadata layer in files like PDFs or Word documents, which can contain semantically identifying fields such as author names, creation dates, and titles. It is a critical first step because metadata often exists in the surrounding system (e.g., file systems or emails) and can unintentionally reveal personal information even if the main text appears anonymous. (Krebs & Hagenweiler, 2022, pp. 107–110)

Content-Level Masking and Natural Language Processing This technique uses Natural Language Processing (NLP) to identify specific entities within a text—such as names, companies, or locations—and replaces them with placeholders or generic strings. Because manual redaction is often inefficient for large datasets, computer-linguistic methods like named entity recognition are used to automatically extract and mask these identifiers. (Krebs & Hagenweiler, 2022, pp. 107–110)

Paraphrasing and Coreference Replacement This method replaces identifying references with generic descriptions or alternative phrases based on linguistic resources like ontologies or word embeddings. By identifying ”coreferences” (repeated mentions of the same person), researchers can replace specific names with broader categories (e.g., changing a specific street to ”a residential area”) while maintaining the semantic flow and readability of the document. (Krebs & Hagenweiler, 2022, pp. 107–112)

Author Obfuscation This technique aims to mask an individual's unique writing style, which can otherwise be identified through digital text forensics. Advanced approaches like Adversarial Author Attribute Anonymity Neural Translation (A4NT) use neural networks to ”translate” a text into an identity-masking form that hides attributes like gender or age while preserving the original meaning. (Krebs & Hagenweiler, 2022, pp. 110–112)

Multimedia Processing Techniques

Multimedia processing techniques are specialized methods used to obscure personal identifiers within audio, image, and video data to prevent the re-identification of individuals by human observers or automated systems. For visual media, standard approaches include pixelation (mosaicking), which divides sensitive image areas into blocks filled with average color values, and blurring, often implemented via Gaussian filters to hide faces or license plates. More direct methods include ”blanking” (cutting out regions of interest) or the use of black bars, though these can often destroy the analytical context of the background. Advanced AI-driven techniques utilize digital avatars or a GAN to replace original facial features or bodies with synthetic versions that preserve the utility of movements and gestures without revealing identity. For audio data, protection involves masking unique vocal characteristics through filtering, pitch shifting, or noise addition, though comprehensive protection must also account for identifying linguistic patterns and word choices. Ultimately, the effectiveness of these techniques depends on the strength of the transformation and the surrounding context, as re-identification remains a risk if environmental clues persist or if the distortion is insufficient to resist modern biometric algorithms. (Krebs & Hagenweiler, 2022, pp. 112–117) (GMDS, b., BvD, 2024, pp. 64–65)

6.4 The Risk of Re-Identification

There is no fixed, universally applicable threshold that, once met, automatically renders data anonymous; instead, anonymization is a contextual state determined by a ”reasonability test”. Because absolute anonymization — where re-identification is impossible under any circumstances — is considered practically unachievable and is not legally required, the focus is on whether identification is unlikely or requires a disproportionate effort in terms of time, cost, and manpower. Consequently, rather than relying on a static limit, researchers must perform a comprehensive risk assessment that considers the specific data environment, the intended use-case (such as internal analysis versus public release), and the likely means available to a potential attacker. This evaluation should be formalized in a risk report or ”attacker model” that documents the chosen transformation methods, identifies potential re-identification scenarios like ”singling out” or ”linkage attacks,” and quantifies the remaining residual risk. Under the accountability principle of the GDPR, this systematic process of evaluation and documentation is essential to demonstrate that the risk to the rights and freedoms of individuals has been effectively mitigated. (Krebs & Hagenweiler, 2022, pp. 120, 124–132) (GMDS, b., BvD, 2024, pp. 18–19, 55, 60–61, 68–71) (Schwartmann et al., 2022, pp. 5–6, 17, 25–26, 28–29, 60–61)

6.5 Deciding on an Approach

Direct identifiers should be pseudonymized by hashing them into deterministic UUIDs. To achieve k-anonymity, data points can be aggregated according to the organizational hierarchy, specifically by grouping employees into their respective teams. However, indirect identifiers, such as Git commit messages, still present a re-identification risk depending on the specific metrics being analyzed; in a worst-case scenario, these messages must be completely removed. Finally, re-identification risks associated with timestamps can be mitigated by injecting temporal noise.

6.6 Implementing the Anonymization Service

To isolate the handling of sensitive data, the anonymization service operates independently from the primary MECOIS architecture. In addition to performing anonymization, the service acts as a proxy for requests directed to SortingHat, granting fine-grained control over the information exposed to MECOIS. Furthermore, this decoupled architecture supports flexible deployment models: individual components can be deployed independently, allowing MECOIS to run as a standalone application or as a fully integrated on-premise installation.

6.6.1 Modules

The anonymization service is structured into five core modules:

`client.requestor` A client library that provides high-level functions to issue HTTP requests to the anonymization server and process incoming responses.

`server.anonymizer` Responsible for content transformation. It either processes payloads directly according to configured anonymization rules or delegates identity-related queries to SortingHat.

`server.server` Encapsulates base server operations, including request initialization, body parsing, and route handling. Upon successful validation of incoming payloads, it delegates processing tasks to the `server.anonymizer` module.

`utils.converter` Since MECOIS relies on PySpark DataFrames for data processing (The Apache Software Foundation, 2026), DataFrames and their schemas must be serialized to JSON for HTTP transmission. This module provides utility functions for serialization and deserialization.

`utils.sortinghat_requestor` A specialized client interface designed to communicate directly with the SortingHat API.

6.6.2 Implementing the Server

The anonymization service is implemented as a RESTful web service to provide a standardized, stateless, and scalable interface within the MECOIS architecture. Following the fundamental principles of Representational State Transfer, the service utilizes a uniform interface where resources are identified by unique URLs and interacted with through standard HTTP methods. To minimize external software dependencies during the prototyping phase, the server is built using Python's native `http.server` module (Python Software Foundation, 2026b). Table 6.1 outlines the endpoints exposed by the server, categorized by HTTP request method. Figure 6.1 represents how the server handles each request. Communication between components is handled via JSON payloads, with the `utils.converter` module ensuring the integrity of PySpark `DataFrames` and their associated schemas during transmission. By maintaining statelessness, the API allows multiple service instances to be scaled horizontally, meeting the performance demands of high-dimensional metadata processing without compromising data consistency.

Table 6.1: Server Endpoints Categorized by HTTP Method

HTTP Method	Endpoints

GET	`docker-compose-healthcheck`
	`ping`
	`show`
	`show-profile`
	`find-profile`
	`find-profiles-by-organization`

POST	`change_identities` `update_identities`

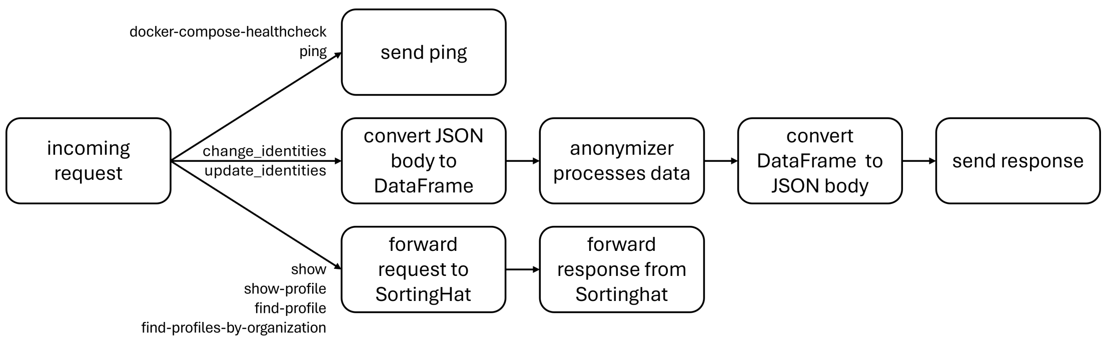

Figure 6.1: Server Diagram

`docker-compose-healthcheck` Provides a health-check endpoint queried periodically by Docker Compose during containerized deployment (see Section 6.6.5).

`ping` Offers a basic connectivity check to verify server availability.

`show` Proxies requests to SortingHat to retrieve the most recent UUID corresponding to a specified identifier.

`show-profile` Forwards requests to SortingHat to fetch full profile metadata associated with a given UUID.

`find-profile` Queries SortingHat for user profiles matching a given search term.

`find-profiles-by-organization` Queries SortingHat for user profiles associated with a specified organization.

`change_identities` Executes pseudonymization and anonymization on the data set (see Sections 6.6.3 and 6.6.4). This endpoint is invoked directly during the Bronze-to-Silver pipeline execution (see Section 5.1.2).

`update_identities` Re-evaluates and updates previously anonymized and pseudonymized datasets in response to upstream modifications within SortingHat. This endpoint is triggered during Silver-to-Silver pipeline operations (see Section 5.1.2).

6.6.3 Implementing the Pseudonymization

Identity management is handled by SortingHat, which assigns a UUID to each identity. The anonymization service subsequently replaces all direct identifiers — specified via a configuration file — with this assigned UUID.

SortingHat generates the UUID by hashing the user's name, email address, and username using the SHA-1 algorithm provided by Python's hashlib library (CHAOSS, 2026b). However, this approach presents cryptographic vulnerabilities: SHA-1 is susceptible to hash collisions, lacks resistance to brute-force attacks, and does not support salting (Python Software Foundation, 2026a). Despite these security limitations, SortingHat's native UUID mechanism is retained during the current development phase due to its implementation simplicity, with a cryptographically secure scheme planned for future iterations.

6.6.4 Implementing the Anonymization

SortingHat supports associating individuals with specific enrollments, where an enrollment is defined by a start timestamp, an end timestamp, and an organization. This capability is leveraged to enrich the DataFrame with an additional column representing each individual's organizational affiliation.

6.6.5 Deploying the Server

For the deployment of the server, this project utilizes Docker Compose, a tool specifically designed for managing multi-container applications through a unified YAML configuration file. This approach offers significant advantages, most notably the ability to link multiple independent microservices and initiate them simultaneously with a single command, providing a higher-level abstraction than individual run commands. Furthermore, the use of Docker ensures consistent environments and application isolation, which prevents dependency conflicts and eases the transition from development to production. The data and application logic are inherently more secure in this setup because they are encapsulated within a container — an isolated instance that utilizes namespaces and its own local file system to remain distinct from the host machine and other services. (Jangla, 2018, pp. 3–5, 13, 28–29, 34–36, 77–81)

7 Evaluation

Following the design and implementation of the anonymization service prototype, this chapter provides a systematic evaluation of the system against the functional and nonfunctional requirements established in Chapter 4. Ultimately, this evaluation determines the extent to which the service provides a legally and technically sound basis for measuring developer productivity in a research or production environment.

7.1 Functional Requirements

FLR-1: Data Anonymization

Evaluating the quality of an anonymization process requires a multifaceted analysis that balances technical robustness, legal compliance, and data utility. The ”goodness” of the resulting dataset is primarily measured through formal anonymity models — such as k-anonymity — which provide mathematical metrics for the degree of protection against identity and attribute disclosure. Beyond these syntactic criteria, the effectiveness of the transformation must be validated against the three fundamental risk scenarios: singling out (isolating an individual), linkability (connecting records across datasets), and inference (deducing sensitive values from other attributes). A robust evaluation further involves the implementation of an attacker model, which simulates realistic adversaries with varying levels of background knowledge and resources to quantify the remaining residual risk. From a legal perspective, the anonymization is considered successful if it achieves factual anonymity, meaning that re-identification is either impossible or requires a disproportionate effort in terms of time, cost, and manpower. Finally, the quality of the process is intrinsically linked to data utility; a high-quality anonymization must minimize information loss to ensure that the dataset remains suitable for the intended analytical purpose while maintaining individual privacy. (Krebs & Hagenweiler, 2022, pp. 73–75, 81–82, 86–88, 103–106, 120–121) (GMDS, b., BvD, 2024, pp. 61–63) (Schwartmann et al., 2022, pp. 3–6, 28–29)

A comprehensive quantitative evaluation of every potential re-identification scenario or the implementation of advanced privacy models such as l-diversity and t-closeness is beyond the scope of this thesis. Instead, the prototype focuses on establishing a robust baseline for privacy by first pseudonymizing direct identifiers, which is achieved by hashing them into deterministic UUIDs within the identity management system. Building upon this layer of protection, k-anonymity is achieved through the aggregation of data points based on organizational hierarchy. This is specifically realized by grouping individuals by their enrollment, a mechanism that links contributors to specific organizations over defined time intervals. By organizing data into these equivalence classes, the service ensures that each record becomes indistinguishable from at least k−1 other individuals, thereby effectively mitigating the risk of singling out specific developers while preserving the utility of group-level productivity metrics.

FSR-1: Preserve Data

The evaluation of FSR-1: Preserve Data reveals an inherent technical and legal conflict with the requirement for data anonymization (FLR-1), as the absolute preservation of all source data is impossible when specific attributes must be transformed or removed to protect individual privacy. In the current prototype, this requirement is addressed through a tiered approach: direct identifiers are pseudonymized by hashing them into deterministic UUIDs, which remain resolvable through the SortingHat identity management system to maintain longitudinal traceability. To further protect privacy, individual contributors are generalized into equivalence classes based on their organizational enrollment, effectively achieving k-anonymity. While the retention of pseudonymized data is permissible and necessary during the scientific research and development phase of MECOIS, it may be mandatory to irreversibly delete the mapping ”keys” in the final product to ensure compliance with the GDPR's anonymization standards. Consequently, the preservation of data utility remains dynamic; indirect identifiers, such as Git commit messages, may also be subject to future deletion if they pose a disproportionate re-identification risk.

FSR-2: Independent Deployment

The prototype successfully achieves architectural and operational decoupling from the primary MECOIS framework. By design, the anonymization service operates as an isolated system that communicates via standard HTTP interfaces, allowing it to be deployed as a standalone component or as part of a fully integrated on-premise installation. From a security perspective, enclosing sensitive data within these independent systems establishes a strict isolation boundary, thereby reducing the risk of unauthorized data exposure and preventing sensitive payloads from leaking into neighboring host components or the primary analytics engine.

This technical independence is primarily realized through the use of Docker Compose, which encapsulates the service and its specific environment within isolated containers, thereby preventing dependency conflicts and ensuring consistent behavior across different deployment environments. While the service functions as a proxy for the SortingHat, this decoupled architecture provides the necessary flexibility to scale the anonymization layer independently of the main analytics engine to meet varying performance demands.

FSR-3: Deterministic Pseudonym Generation

The prototype successfully prioritizes a deterministic approach to ensure the consistency of longitudinal data analysis. By utilizing the native UUID generation mechanism within the SortingHat identity management system — which hashes direct identifiers such as names, emails, and usernames — the service ensures that the same individual is assigned a consistent pseudonym across different pipeline runs. This determinism is essential for the MECOIS framework's goal of linking cross-platform metrics from disparate sources like GitHub and Jira without revealing the developers' actual identities. However, the implementation is currently constrained by the use of the SHA-1 hashing algorithm, which presents cryptographic vulnerabilities such as a lack of resistance to brute-force attacks and an absence of salting. While this simplified method is functionally sufficient for the prototype's current research phase and supports the necessary traceability, the evaluation highlights that a more secure, salted hashing scheme will be required for the final product to more effectively mitigate re-identification risks.

FIR-1: Seamless Integration

The anonymization service successfully integrates into the MECOIS framework by replacing legacy processing steps with calls to the new server. Specifically, the previous, insufficient approach to anonymization by pseudonymizing direct identifiers in the Bronze-to-Silver pipeline has been replaced by invoking the `change_identities` endpoint, ensuring that the data is anonymized as an integral part of the data flow. Similarly, the Silver-to-Silver pipeline has been adapted to utilize the `update_identities` endpoint to maintain data consistency when changes occur within the identity management system. Furthermore, existing communication with the SortingHat system has been successfully redirected through the new service, which now acts as a proxy for requests such as `show-profile` or `find-profile`. This transition fulfills the requirement for minimal disruption to existing features while providing the system with fine-grained control over the personal information exposed to the analytics engine.

7.2 Nonfunctional Requirements

NFLR-1: Anonymization Methodology Adherence

The prototype strictly implements the specific technical measures established during the design phase in Chapter 6. The service correctly operationalizes the hybrid strategy of combining deterministic pseudonymization with structural aggregation to balance analytical utility and developer privacy. Specifically, direct identifiers are replaced with stable pseudonyms generated via the SortingHat identity management system, ensuring that longitudinal analysis remains possible across disparate data sources like Git and Jira. Furthermore, the service enforces the decided approach for k-anonymity by generalizing individual contributors into equivalence classes based on their organizational enrollment and team affiliations. Consequently, the evaluation confirms that the system adheres to the defined methodology, providing a robust and legally grounded foundation for the MECOIS Productivity Index.

NFSR-1: Containerized Service

The requirement is fully met through the implementation of `Docker Compose`, which manages the service as part of a multi-container application via a unified YAML configuration. This containerized approach ensures environmental consistency and application isolation, effectively preventing dependency conflicts that could arise during the transition from development to production. Furthermore, the encapsulation of the application logic within a container provides a secure, isolated instance that utilizes its own namespaces and local file system to remain distinct from the host machine and other integrated services. By leveraging these containerization technologies, the prototype achieves a high-level abstraction that allows for the simultaneous initiation of all necessary microservices with a single command, fulfilling the demand for a portable and robust deployment model.

NFSR-2: RESTful API

The MECOIS framework is provided with a standardized and scalable communication interface. The anonymization service adheres to the core principles of Representational State Transfer by utilizing unique URLs to identify resources and standard HTTP methods for interaction, which ensures a clear separation of concerns between the main system and the identity management layer. By maintaining a stateless architecture, the service processes each request independently without relying on stored session context, thereby facilitating efficient horizontal scalability. This design allows the anonymization service to act as a transparent proxy for the SortingHat API while remaining operationally decoupled from the primary analytics engine. Furthermore, the implementation leverages JSON payloads to ensure data integrity during the transmission of high-dimensional metadata, satisfying the need for an interoperable and robust integration within the decentralized environment.

NFIR-1: Data Integrity

The anonymization service effectively maintains the consistency of data structures during its integration with the broader MECOIS framework. This requirement is successfully addressed by the `utils.converter` module, which manages the serialization and deserialization of `PySpark DataFrames` and their corresponding schemas into JSON format for HTTP transmission. By preserving schema integrity, the system ensures that no data loss occurs during the communication between the decentralized system components, thereby maintaining the quality of the activity-based artifacts required for statistical analysis. This technical measure is critical for the reliable derivation of the Productivity Index, as it ensures that the high-dimensional metadata remains structurally sound and analytically useful throughout the entire anonymization pipeline.

NFIR-2: Programming Language

The requirement is fully satisfied, as the anonymization service is implemented entirely in `Python3`. This alignment with the existing MECOIS backend ensures seamless interoperability between the system components, particularly for the processing of `PySpark DataFrames` and the integration of the service into the multi-stage data pipeline. Furthermore, the use of `Python` allows the prototype to leverage standardized libraries, such as `http.server` for the backend implementation and `hashlib` for generating deterministic UUIDs, while adhering to the project's mandated coding and license policies. By fulfilling this requirement, the service maintains technical consistency across the research framework, facilitating easier maintenance and future development.

NFIR-3: License Policy

The anonymization service adheres to the project's mandate to avoid the use of packages with copyleft or proprietary licenses. The prototype is implemented using `Python` and relies primarily on its standard libraries, such as `http.server` for the backend, which conform to permissive licensing standards. Furthermore, while the service interfaces with the SortingHat identity management system—which is governed by a `GPL-3.0` license — it maintains a strictly decoupled relationship by communicating exclusively via `HTTP`. This architectural separation ensures that the service's internal codebase remains independent of external copyleft requirements, thereby fulfilling the integration constraints established by the MECOIS contributing guidelines.

NFIR-4: Styleguide and Clean Code

The implementation of the anonymization service strictly adheres to the `MECOIS` contributing guidelines to ensure high software quality and long-term maintainability. The codebase is developed in alignment with the `PEP8` standard. By fulfilling these requirements, the service ensures technical consistency with the rest of the `MECOIS` backend, facilitating a seamless review and merging process into the main branch.

8 Conclusions

This thesis has presented the design, implementation, and evaluation of a prototype data anonymization service for the MECOIS framework, that aligns the software engineering productivity analytics with individual privacy rights under the GDPR and BDSG. By integrating this service as a decoupled layer within the MECOIS architecture, the project demonstrates a viable path for organizations to derive actionable insights from developer metadata while maintaining a privacy-first approach.

8.1 Summary of Results

The developed prototype successfully fulfills the core functional and nonfunctional requirements established for a GDPR and BDSG-compliant benchmarking system. Key achievements of this work include:

Architectural Independence The service operates as a stateless, containerized system, allowing for independent deployment and horizontal scalability without interfering with the primary MECOIS analytics engine.

Privacy Mechanisms The service establishes a robust baseline for privacy by pseudonymizing direct identifiers into deterministic UUIDs and achieving k-anonymity through the aggregation of contributors into organizational equivalence classes.

Seamless Integration By acting as a proxy for the SortingHat identity management system and providing dedicated endpoints for identity transformation, the service integrates into the existing Bronze-to-Silver and Silver-to-Silver pipelines with minimal disruption.

Data Integrity Through the `utils.converter` module, the service ensures that PySpark DataFrames and their schemas are preserved during HTTP transmission.

8.2 Discussion and Legal Compliance

From a legal perspective, the prototype enables the project to evolve from scientific research to a final product. While the current use of pseudonymization is sufficient for the research phase of MECOIS, the service provides the necessary infrastructure to transition toward factual anonymization, where re-identification requires disproportionate effort, by generalized grouping and the potential permanent deletion of re-identification keys. The evaluation confirms that the system effectively mitigates the risk of ”singling out” individuals through its implementation of equivalence classes based on organizational enrollment.

8.3 Future Work

Although the current prototype fulfills the core requirements for the MECOIS framework, several potential enhancements regarding the service’s security, privacy robustness, and production readiness remain. These refinements, including the implementation of advanced privacy models and a comprehensive quantitative evaluation of re-identification scenarios, were not executed as part of this project as they fall beyond the defined scope of this thesis.

Assuring k-anonymity

While the current prototype achieves a baseline for k-anonymity by grouping contributors into organizational equivalence classes, it lacks an automated mechanism to enforce the k threshold. Future iterations must implement a check to ensure that every organization in a dataset contains at least k enrolled individuals. If this threshold is not met, the system should automatically aggregate smaller organizations into broader clusters to maintain the necessary anonymity level. Furthermore, the system must account for temporal risks such as employee vacations; if a query targets a narrow timeframe where several contributors are inactive, the number of actual records may fall below the k limit even if the total enrollment is sufficient. Implementing dynamic checks for active record counts within queried intervals is essential to prevent re-identification through ”singling out”.

Permission System and Logging

The current service acts as a proxy for the SortingHat identity management system, but it lacks a formal access control layer. Currently, any component within the network can query the system to resolve identities. To align with GDPR accountability standards, a robust permission system and authentication layer must be implemented to restrict who can resolve UUIDs or access sensitive profile metadata. Additionally, every query to the identity management system should be logged to provide a transparent audit trail of how and when personal data is processed.

Production-Grade Server

The prototype utilizes Python's built-in `http.server` module because it minimizes dependencies and requires no complicated setup. However, this module is not recommended for production environments as it only implements basic security checks and lacks the performance capabilities for high-volume data pipelines. Future versions should replace this with a production-grade framework such as Flask combined with Gunicorn and Nginx. This upgrade would provide enhanced security, support for multiple threads, and asynchronous execution, ensuring the service can scale horizontally to handle heavy processing loads behind a load balancer. (Python Software Foundation, 2026b) (ancisoft.com, 2026)

Anonymization Analysis

A critical next step is a formal, comprehensive analysis of the anonymization effectiveness. While the prototype implements key privacy strategies, a complete report is needed to document the "reasonability" of the approach, evaluating whether re-identification requires a disproportionate effort in terms of time and cost. This analysis should be formalized as an "attacker model" that quantifies residual risks such as linkage or inference attacks based on the specific developer metadata processed by MECOIS.

Cryptographic Security

The service currently relies on SortingHat's native UUID generation, which uses a SHA-1 hashing mechanism. As noted in the evaluation, SHA-1 is susceptible to hash collisions and lacks resistance to brute-force or rainbow table attacks. To improve cryptographic security, future iterations must implement a more secure, salted hashing scheme. Appending a "salt" — a random sequence of characters — before hashing will significantly increase the entropy of the identifiers and prevent attackers from using precomputed tables to reverse the transformation. (Python Software Foundation, 2026a)

Unstructured Data Processing

While the service successfully handles structured identifiers, indirect identifiers within unstructured data remains a significant challenge. Specifically, Git commit messages often contain timestamps, specific URLs, or linguistic patterns that could lead to re-identification. Future work should integrate advanced NLP for content-level masking or author obfuscation techniques, such as A4NT, to mask unique writing styles while preserving the analytical utility of the messages.

Employee Representation Bodies

Various supervisory and representative bodies exist to safeguard employee rights. Consequently, the deployment and operation of a tool like MECOIS must be closely coordinated with these bodies to satisfy statutory and organizational requirements.

Data Protection Officer Under certain statutory conditions, organizations are obligated to designate a Data Protection Officer (DPO). The DPO's responsibilities encompass providing advisory support, monitoring compliance, and liaising with the supervisory authority. Furthermore, the DPO must inform and advise controllers of their legal obligations under data protection law and audit compliance with statutory mandates, such as the GDPR. Consequently, the DPO must be integrated at an early stage during the planning and design of future digital forms of work (Däubler, 2017, pp. 392–394, 400–402).

Works Council Corporate deployment of MECOIS requires mandatory codetermination by the works council (*Betriebsrat*) under § 87 Abs. 1 Nr. 6 BetrVG, as processing granular activity artifacts (e.g., timestamps, commit details) constitutes technical employee monitoring. Beyond statutory information obligations (§ 80 Abs. 2 BetrVG) and expert consultation rights regarding AI-driven analytics (§ 80 Abs. 3 BetrVG), deployment necessitates a binding shop agreement (*Betriebsvereinbarung*). The prototype's privacy controls, specifically deterministic UUIDs and k-anonymity via team aggregation, serve as an essential technical foundation for these negotiations to balance productivity analysis with worker protection. (Däubler, 2017, p. 429) (Niedenhoff, 2024, pp. 2–6, 61–62, 72–73, 79–81, 96–97, 99–101, 134, 348–351)

Right to Erasure and Deletion Cascade

To ensure full compliance with Art. 17 GDPR, commonly referred to as the "Right to be Forgotten," the service must implement a mechanism capable of executing a deletion cascade upon a contributor's request. This process requires the system to irreversibly purge or "hard-anonymize" all historical mappings, pseudonym keys, and logs that allow for the re-identification of that specific individual. While the legal equivalence of anonymization and deletion is a subject of academic debate, a robust transformation that renders re-identification impossible—or possible only through a disproportionate effort in time and cost — can satisfy the requirement to make personal data unrecognizable. Within the

SortingHat or similar identity management systems, this necessitates the permanent destruction of deterministic UUID mapping tables to break the link between the pseudonym and the original direct identifiers. Furthermore, achieving absolute anonymity requires that no party, including the original controller, retains the means to resolve these identifiers. Transitioning the research prototype into a legally compliant product therefore depends on the implementation of these irreversible erasure protocols to uphold the accountability principle and protect the data subject's rights. (Krebs & Hagenweiler, 2022, pp. 71–72, 126–130) (GMDS, b., BvD, 2024, pp. 13, 18–20, 73–75) (Schwartmann et al., 2022, pp. 13–17)

8.4 Concluding Remarks

In summary, the rapid expansion of Big Data analytics continues to serve as a primary driver of innovation across global industries, yet this progress is frequently at odds with the fundamental right to data privacy. This thesis has addressed this critical tension by providing a scalable and legally grounded foundation for the MECOIS framework, demonstrating that high-dimensional software engineering metadata can be harnessed for productivity insights without violating the mandates of the GDPR and BDSG. By implementing a decoupled, containerized anonymization service, this work contributes a robust methodology for transforming sensitive developer artifacts into privacy-preserving metrics through pseudonymization and k-anonymity. This prototype effectively bridges the gap between academic research and the requirements of a final, production-ready product by establishing the necessary infrastructure for factual anonymization. Ultimately, the results of this study ensure that the pursuit of enhanced team productivity does not come at the expense of individual developer privacy, offering a viable path forward for ethical, data-driven analytics in modern software development environments.

Appendices

A Declaration on the Use of Artificial Intelligence

I declare that Large Language Models (Google Gemini 3.6, OpenAI ChatGPT 5.5, and Google NotebookLM) were utilized during the preparation of this Master's thesis. Their application was strictly limited to language editing, refining sentence structure, polishing style, and correcting grammar across selected chapters.

No core concepts, original arguments, research methodologies, data analyses, or final conclusions were generated by these tools. All literature cited was independently identified and verified, and I remain fully responsible for the original content, accuracy, and integrity of this work.

References

Amazon Web Services. (2026). *What is a data lakehouse?* Amazon Web Services. Retrieved July 29, 2026, from https://aws.amazon.com/what-is/data-lakehouse/

ancisoft.com. (2026). *Why use gunicorn + nginx with flask? benefits explained vs running flask alone*.

Buchner, S., & Riehle, D. (2021). Calculating the costs of inner source collaboration by computing the time worked. https://dirkriehle.com/wp-content/uploads/2021/09/Buchner_Riehle__LOCAL-VERSION__Computing_the_Costs_of_Inner_Source_Collaboration.pdf

CHAOSS. (2026a). *Profiles and identities*. GrimoireLab. Retrieved July 14, 2026, from https://chaoss.github.io/grimoirelab-tutorial/docs/sortinghat/profiles/

CHAOSS. (2026b). *Sortinghat github repository*. GrimoireLab. Retrieved July 14, 2026, from https://github.com/chaoss/grimoirelab-sortinghat

CHAOSS. (2026c). *Sortinghat's interface*. GrimoireLab. Retrieved July 14, 2026, from https://chaoss.github.io/grimoirelab-tutorial/docs/sortinghat/interface/

Däubler, W. (2017). *Gläserne belegschaften: Das handbuch zum beschäftigtendatenschutz* (7., a thoroughly revised and updated edition). Bund-Verlag.

Dueñas, S., Cosentino, V., Gonzalez-Barahona, J. M., del Castillo San Felix, A., Izquierdo-Cortazar, D., Cañas-Díaz, L., & Pérez García-Plaza, A. (2021). Grimoirelab: A toolset for software development analytics. *PeerJ Computer Science*, *7*, e601. https://doi.org/10.7717/peerj-cs.601

für den Datenschutz und die Informationsfreiheit, D. B. (2025). *Die europäische datenschutz-grundverordnung (dsgvo)*. Retrieved July 19, 2026, from https://www.bfdi.bund.de/DE/BfDI/Inhalte/Datenschutzpfad/DSGVO.html

Iso/iec/ieee international standard - systems and software engineering–vocabulary. (2017). *ISO/IEC/IEEE 24765:2017(E)*, 1–541. https://doi.org/10.1109/IEEESTD.2017.8016712

Jangla, K. (2018). *Accelerating development velocity using docker: Docker across microservices* [Library of Congress Control Number: 2018962734]. Apress. https://doi.org/10.1007/978-1-4842-3936-0

Krebs, H.-A., & Hagenweiler, P. (2022). *Datenanonymisierung im kontext von künstlicher intelligenz und big data: Grundlagen – elementare techniken – anwendung*. Springer Vieweg. https://doi.org/10.1007/978-3-658-37588-1

MECOIS. (2026). *Mecois github repository*. MECOIS. Retrieved July 8, 2026, from https://github.com/Mecois/mecois

Members of the "Deutsche Gesellschaft für Medizinische Informatik, " d. D. D. e. V. (, Biometrie und Epidemiologie e. V." (GMDS) research group "Datenschutz und IT-Sicherheit im Gesundheitswesen", e. V." (bvitg) research group "Datenschutz, " G.-I., & IT-Sicherheit". (2024, January 27). *Praxishilfe zur anonymisierung/pseudonymisierung* (Praxishilfe). Berlin/Bonn.

Niedenhoff, H.-U. (2024). *Betriebsräte in deutschland: Vom arbeiterausschuss zum mitgestalter des arbeitslebens*. Springer Gabler. https://doi.org/10.1007/978-3-658-44226-2

Python Software Foundation. (2026a). *Hashlib — secure hashes and message digests: Python 3.14.6 documentation* (3.14.6). Python Software Foundation. Retrieved July 22, 2026, from https://docs.python.org/3/library/hashlib.html

Python Software Foundation. (2026b). *Http.server — http servers: Python 3.14.6 documentation* (3.14.6). Python Software Foundation. Retrieved July 23, 2026, from https://docs.python.org/3/library/http.server.html

Riehle, P. D. D. (2025). *Mecois - productivity measurement*. University of Erlangen, Professorship for Open-Source Software. Retrieved July 19, 2026, from https://www.mecois.com/solutions/productivity-measurement

Schwartmann, R., Jaspers, A., Lepperhoff, N., Weiß, S., & Meier, M. (2022, December). *Praxisleitfaden zum anonymisieren personenbezogener daten: Anforderungen, einsatzklassen und vorgehensmodell* (Praxisleitfaden). Stiftung Datenschutz. Berlin.

Selvaraj, S. (2024). *Mastering REST APIs: Boosting your web development journey with advanced API techniques* (1st). Apress. https://doi.org/10.1007/979-8-8688-0309-3

Stenzel, M. (2026). Estimating software contribution time from git and github metadata. https://oss.cs.fau.de/wp-content/uploads/2026/04/Stenzel_2026.pdf

The Apache Software Foundation. (2026). *Quickstart: Dataframe* (4.2.0). The Apache Software Foundation. Retrieved July 23, 2026, from https://spark.apache.org/docs/latest/api/python/getting_started/quickstart_df.html